Assignment 4 - Dynamic Programming and String Processing

Edition 2.1.0. Use this GitHub template for your implementation; submit the exact commit link in Moodle.

Course: Design and Analysis of Algorithms. **Assessment:** practical work 50 + individual defence quiz 50 = 100; Assignment 4 contributes 20% of the second attestation. Quiz II is a separate assessment. **Deadline:** as announced in Moodle.

1. Objective and syllabus scope

Implement six parts with eight request commands: LCS, ordered merging, tree selection, three single-pattern searches, suffix indexing and dictionary matching. They are independent modules for sequence comparison, planning and text processing; they do not share application state.

Weeks 8-10 cover subsequence, partitioning and tree DP; KMP and polynomial rolling hashing; suffix arrays, Kasai LCP and Aho-Corasick. Fibonacci and 0/1 Knapsack are preparation exercises, not additional required implementations. The merge task applies the interval-partition pattern from lecture 8; its cost rule differs from matrix-chain multiplication.

Java 17+ is the syllabus reference language. Choose one of the supported course languages: Java 17+, Python 3.12+, C++ 17 or C#/.NET 8. Standard collections, sorting and supplied JSON transport are allowed. Implement the required algorithms yourself; an algorithm library or built-in substring-search routine may not replace them.

2. Shared program contract

Keep the supplied UTF-8 JSON Lines interface: one request object per input line and one result object per output line. Write only results to stdout. A process may handle consecutive requests with different tasks; reset task state for every request. Strings contain printable ASCII codes 32-126, are case-sensitive and have no Unicode normalization. Spaces and punctuation are ordinary characters.

All character positions, block indices and vertex IDs are zero-based. Required numeric fields in the input/output are integers, not JSON Booleans or fractional numbers. Costs, total weights and occurrence counts must be represented exactly with signed 64-bit-safe integer arithmetic; Python integers are suitable. Preserve input array order where it defines output positions. JSON object-field order is unrestricted. Inputs satisfy the published domain; malformed JSON is not assessed. A different valid optimum is accepted wherever reconstruction is non-unique.

The field signatures below use type notation; the worked examples show literal valid JSON. Each output must contain the required named fields with their stated types, even when only an optimum-value group is being checked. Extra object fields are ignored by the current validators; they are not required.

Complexity bounds concern the algorithm after JSON parsing. Report auxiliary storage separately from input/output storage. For string search n is text length, m is pattern length and z is the number of returned starts; emitting z positions requires at least z output operations. Handle empty inputs explicitly.

There are six parts, eight commands and nineteen practical marking groups. Part 4 contains naive, kmp and rk; suffix includes both suffix-array construction and Kasai LCP.

Part 1: Compare two sequences - LCS

Compare two versions of a sequence while preserving the order of shared characters.

Command: `lcs`

Input: `{task:lcs,a:string,b:string}`

Output: `{length,sequence:string}`

Limits: 0 <= |a|,|b| <= 1000; |a|*|b| <= 1000000.

Implement prefix dynamic programming and reconstruct one longest common subsequence. sequence must be a subsequence of both input strings and have the optimal length. It need not be contiguous. Any optimal sequence is accepted. If either input string is empty, return length=0 and sequence="". Explain states, transitions, base cases and reconstruction in ANALYSIS.md.

Complexity: O((|a|+1)(|b|+1)) time and memory are permitted, including empty inputs.

Example input:

```
{
  "task": "lcs",
  "a": "tea_time",
  "b": "team"
}
```

One accepted result:

```
{
  "length": 4,
  "sequence": "team"
}
```

The final m in tea_time is separated from the preceding letters: team is a subsequence, not a substring.

Explain: Define the prefix state and explain why the mismatch transition and reconstruction preserve an optimum.

Part 2: Merge ordered blocks - interval DP

Plan the cheapest sequence of adjacent block merges when the original order must be preserved.

Command: `merge`

Input: `{task:merge,sizes:[...]}`

Output: `{cost,plan:[[l,k,r],...]}`

Limits: sizes is an array of n integers; 0 <= n <= 200 and 0 <= sizes[i] <= 1000000000.

A current block represents a closed original interval. Merge adjacent blocks [l, k] and [k+1, r]; pay their combined size and replace them by [l, r]. Return the minimum total cost and a chronological valid plan using original indices. Each input block begins as [i, i]. For n<=1 cost=0 and plan=[]. Any optimal plan is accepted. Greedily choosing the smallest two arbitrary blocks violates adjacency. For n>1 the plan contains exactly n-1 merges.

Complexity: O(n^3) time and O(n^2) auxiliary memory for n>=1; O(1) for no blocks. No advanced DP optimization is required.

Example input:

```
{
  "task": "merge",
  "sizes": [2, 3, 4]
}
```

One accepted result:

```
{
  "cost": 14,
  "plan": [[0, 0, 1], [0, 1, 2]]
}
```

The two merge costs are 5 and 9, giving 14; the other order gives 7+9=16.

Explain: Define the interval state, explain the last split and why the plan must output child merges before their parent merge.

Part 3: Select compatible vertices - tree DP

Choose a maximum-value set when neighbouring vertices are incompatible and conflicts form a tree.

Command: `tree`

Input: `{task:tree,n,weights:[...],edges:[[u,v],...]}`

Output: `{weight,selected:[vertex IDs,...]}`

Limits: n is an integer in 0..100000. weights has exactly n integer entries; -1000000000 <= weights[i] <= 1000000000. For n>0, edges contains n-1 distinct undirected edges forming a connected tree; every endpoint is an integer in 0..n-1. For n=0, weights and edges are empty. The output total weight is not limited to the bound on one entry.

Implement maximum weight independent set by take/skip tree DP and reconstruct a selected set. The empty set is allowed, so all-negative inputs return weight 0. No adjacent vertices may both be selected. Any optimum is accepted. Root choice must not change the optimum; avoid recursion failure on deep chains. Each selected vertex must occur once; order is unrestricted. For n=0 return weight=0 and selected=[].

Complexity: O(n+1) time and auxiliary memory, including reconstruction; the empty tree takes O(1).

Example input:

```
{
  "task": "tree",
  "n": 3,
  "weights": [4, 9, 6],
  "edges": [[0, 1], [1, 2]]
}
```

One accepted result:

```
{
  "weight": 10,
  "selected": [0, 2]
}
```

Vertices 0 and 2 are nonadjacent and contribute 4+6=10; the middle vertex alone contributes 9.

Explain: How take/skip and postorder work, and how the parent decision restricts reconstruction.

Part 4: Exact search - naive baseline

Establish a simple exact-search baseline against which the other two searches can be checked.

Command: `naive`

Input: `{task:naive,text:string,pattern:string}`

Output: `{positions:[...]}`

Limits: 0 <= |text| <= 3000; 0 <= |pattern| <= 3000.

Implement naive character comparisons. Return ALL zero-based occurrence starts in strictly increasing order, including overlaps. Empty pattern matches every boundary 0..|text|, including the final boundary. Pattern longer than text returns []. No built-in search functions in the algorithm.

Complexity: With n=|text| and m=|pattern|: O(n*m+n+1) time after parsing, including the output; O(z+1) output memory for z starts and O(1) auxiliary memory. The empty-pattern case outputs n+1 starts.

Example input:

```
{
  "task": "naive",
  "text": "aaaaa",
  "pattern": "aaa"
}
```

One accepted result:

```
{
  "positions": [0, 1, 2]
}
```

All three starts are valid because overlaps count.

Explain: The last valid start, overlaps and empty-pattern boundaries.

Part 4: Exact search - KMP

Reuse information about matched prefixes to avoid restarting the search after every mismatch.

Command: `kmp`

Input: `{task:kmp,text:string,pattern:string}`

Output: `{positions:[...]}`

Limits: 0 <= |text| <= 200000; 0 <= |pattern| <= 100000.

Implement prefix function and KMP. Use exactly the same matching contract as naive, including the empty pattern. Keep overlaps after a full match using prefix fallback. Compare outputs of all three search algorithms on inputs in their common domain. Built-in search may not replace KMP.

Complexity: O(n+m+z+1) time, O(m+1) auxiliary memory and O(z+1) output memory, where n=|text|, m=|pattern| and z is the number of starts. An empty pattern returns all n+1 boundaries before any pattern indexing.

Example input:

```
{
  "task": "kmp",
  "text": "abababa",
  "pattern": "aba"
}
```

One accepted result:

```
{
  "positions": [0, 2, 4]
}
```

Prefix fallback after a match preserves the overlapping starts 2 and 4.

Explain: Define pi[i] and explain fallback after a mismatch and after a full match.

Part 4: Exact search - Rabin-Karp

Reject windows with unequal fingerprints and verify every equal-hash candidate.

Command: `rk`

Input: `{task:rk,text:string,pattern:string}`

Output: `{positions:[...]}`

Limits: 0 <= |text|,|pattern| <= 10000; |text|*|pattern| <= 20000000.

Implement polynomial rolling hash. Use base 31, modulus 101 and ASCII character codes; highest power belongs to the leftmost character. This deliberately small modulus makes collisions testable. Normalize negative remainders. Equal hashes are candidates only: compare actual characters before reporting. The exact matching contract, including empty patterns and overlaps, is shared with KMP. 64-bit arithmetic is sufficient. No built-in search functions.

Complexity: For 1<=m<=n, O(n+m+c*m) time, where c is the number of equal-hash candidate windows; this can be quadratic. A bound covering all permitted inputs is O(n*m+n+m+1), including the n+1 empty-pattern starts. Handle m=0 and m>n before forming a window. O(m+1) auxiliary memory plus output is allowed for rolling windows; O(n+m+1) plus output is allowed with prefix hashes. Do not claim deterministic linear worst-case time.

Example input:

```
{
  "task": "rk",
  "text": "abdj",
  "pattern": "ab"
}
```

One accepted result:

```
{
  "positions": [0]
}
```

ab and dj both hash to 75 with base 31/mod 101; only ab is an actual match.

Explain: The rolling update, collision verification and the effect of candidate windows on running time.

Part 5: Index repeated text - suffix array and Kasai LCP

Order the text suffixes and use adjacent common prefixes to find repeated substrings.

Command: `suffix`

Input: `{task:suffix,text:string}`

Output: `{sa:[...],lcp:[...],repeated:string}`

Limits: 0 <= |text| <= 50000. Build exactly the nonempty suffixes; do not include a sentinel or the empty suffix in output.

Implement prefix doubling and Kasai. sa is a permutation of 0..n-1 in lexicographic suffix order. lcp has length n; lcp[0]=0 and lcp[i] compares sa[i-1] with sa[i]. Return any longest substring occurring at two distinct starts; overlaps are allowed. If none exists, repeated="". For empty text, return sa=[], lcp=[] and repeated="". Do not materialize all suffix strings.

Complexity: For n>=1, prefix doubling with comparison sorting takes O(n log^2(n+1)) time; linear-time counting/radix sorting of rank pairs per round permits O(n log(n+1)). Kasai takes O(n). Auxiliary and output memory are O(n+1). Empty input takes O(1).

Example input:

```
{
  "task": "suffix",
  "text": "banana"
}
```

One accepted result:

```
{
  "sa": [5, 3, 1, 0, 4, 2],
  "lcp": [0, 1, 3, 0, 0, 2],
  "repeated": "ana"
}
```

ana starts at 1 and 3; these occurrences overlap. Its length equals max(lcp).

Explain: Rank pairs, Kasai reuse and why maximum adjacent LCP gives a longest repeat.

Part 6: Search a dictionary - Aho-Corasick

Build one dictionary automaton and count its patterns in several independent texts.

Command: `aho`

Input: `{task:aho,patterns:[nonempty strings,...],texts:[strings,...]}`

Output: `{counts:[[count for each dictionary position],...]}`

Limits: 0 <= number of patterns <= 10000; total pattern length <= 100000; 0 <= number of texts <= 100; total text length <= 200000. Patterns are nonempty; texts may be empty.

Build one Aho-Corasick trie/automaton per request, with suffix/failure links. Query it for each independent text; reset state and counts. Count overlaps and suffix-pattern matches. Duplicate patterns are separate dictionary positions and receive equal counts. An empty dictionary produces an empty count row per text. An empty texts array produces []. Do not replace the automaton by multiple KMP searches. Count results using visit propagation along failure links or correct output-link traversal. Output rows follow texts order; columns follow original dictionary positions. An empty text has one zero per dictionary position. The response is counts, not a list of all individual matches: share duplicate terminal states and aggregate counts rather than creating one record for every dictionary-entry occurrence. For dense

matches, use reverse-failure visit propagation or an equally efficient aggregation; a literal per-match traversal can exceed the time limit despite being logically correct.

Complexity: Let L be total pattern length including duplicates, p dictionary size, and S trie states including the root. With the fixed 95-character ASCII alphabet, construction takes O(L+95*S+p) time and O(95*S+p) auxiliary memory. Reverse-failure propagation per independent text takes O(|text|+S+p). Output-link traversal instead takes O(|text|+z+p), where z counts visited terminal matches; it is not a guarantee of meeting the limits on dense inputs. Output memory is O(q*(p+1)+1) for q texts; retain one empty row per text when p=0. Avoid quadratically copied inherited output lists.

Example input:

```
{
  "task": "aho",
  "patterns": ["he", "she", "he"],
  "texts": ["sheshe", "sheshe", ""]
}
```

One accepted result:

```
{
  "counts": [[2, 2, 2], [2, 2, 2], [0, 0, 0]]
}
```

he is a suffix of she and is counted too. The repeated text starts with fresh counts.

Explain: Failure links, suffix-pattern counts, duplicates and independent texts.

Assessment and defence

Practical work is worth 50 points from independent automated requirement-group checks. The defence is a separate 50-point individual quiz. The required methods, analysis, own tests and benchmark are learning deliverables with no separate hidden manual points or discretionary score deductions. Result tests cannot prove the internal algorithm or authorship; the defence checks individual understanding. See RUBRIC.md for group weights and correction rules.

The English defence contains 15 questions in 25 minutes, one attempt: 10 single-answer questions with four options each and 5 multiple-response questions with five options each. Each question has a raw maximum of 1; the quiz grade is raw/15×50. A correct single answer earns 1; a wrong or blank answer earns 0. The questions apply the methods to short new cases: explaining a transition, checking a witness, finding a counterexample, diagnosing an incorrect implementation, or interpreting complexity.

For a multiple-response question with k correct options (k=2 or 3), every selected correct option earns 1/k and every selected wrong option deducts 1/(2k). Sum the selected weights and floor at 0. With two correct options: +0.50/-0.25. With three: +1/3/-1/6. Selecting all five options therefore earns 0.25 or approximately 0.67 respectively; full credit requires exactly all correct options. This partial-credit policy follows the latest A3 format and is a course assignment rule. It is not specified by the syllabus. No individual oral score recovery is included. A defective question/key or test is corrected consistently for all affected work under course rules.

Work independently during the quiz, without AI or external help. Follow the quiz instructions and accommodations approved by the course. There are three pages of five questions with free navigation. Dates, access and answer-release settings are announced in Moodle. Question keys are not part of the public assignment package.

Analysis, testing and comparison

Complete ANALYSIS.md in English. For each command explain the state/invariant, transitions or updates, base cases, processing order, correctness, reconstruction where applicable, time and memory. Include tests with input, expected output, actual output and the defect they detect. Cover empty inputs, ties, overlaps, large values and consecutive requests as applicable. Do not fabricate runs or measurements.

Compare naive/KMP/RK on identical text/pattern pairs at three increasing sizes within their common input domain: both lengths≤3000 also satisfy all three domains. Use a reproducible generator/seed and record n, m, pattern distribution, runtime version, commands, timing scope, at least three measured runs per method and medians. Check identical correct outputs before timing; exclude compilation and use the same timing boundary for all methods. Report warm-up/JIT treatment and hardware. Cover random and repetitive cases separately; discuss why a modulus 101 collision laboratory does not represent every production hash implementation. A supplied collision pair is a correctness test; timing medians do not prove asymptotic complexity.

Build, check and submit

Start from the official A4 template linked in Moodle. Select one language in submission.json. Implement the corresponding source methods; preserve the provided transport. The documented launcher in the existing A4 template is:

```
python tools/run.py --language Java --setup
python tools/run.py --language Java --check
python tools/run.py --language Java --input examples/lcs.jsonl
```

Replace Java with Python, Cpp or CSharp. The launcher requires Python 3.12+ even for another implementation language. Windows may use py -3. Omit --language to use submission.json. setup checks launch; TODO algorithm bodies are intentionally incomplete. Public checks contain 44 cases and repeat the complete batch twice in one process to check state reset. Passing setup alone does not test your algorithm results.

Commit your selected source, submission.json, own tests under tests/student, completed English ANALYSIS.md and submission notes. Run local/public checks; the submitted implementation must pass public checks. In Moodle Online text submit the exact commit URL, optionally an Actions run URL for the same commit, then click Submit assignment and verify the status. The commit URL contains the complete 40-character SHA and identifies the assessed version. Give the instructor access to a private repository. Follow the Moodle deadline and any approved extension. Public checks provide feedback; they do not calculate or transfer the official practical grade.

Official isolated checks allow 60 seconds for compilation and 20 seconds per requirement-group batch, 1 CPU, 768 MiB container memory (Java heap≤512 MiB),128 processes and at most 8 MiB UTF-8 output. Each group runs as one JSONL batch. A process failure, timeout, or invalid/missing result line can fail the entire batch; a well-formed but wrong result affects its own case. The solution must support the full input domains above; public examples alone do not establish that. Timing medians do not determine a ranking.

Academic integrity and resources

Follow the syllabus. Acknowledge external sources and AI tools in ANALYSIS.md: contribution, material used and your own verification; state if none were used. Take responsibility for correctness, originality and quality. Written style alone does not establish misconduct.

Use syllabus weeks 8-10 and lecture 8 (DP patterns), lecture 9 file (KMP/hash) and lecture 10 file (suffix/Kasai/Aho). Read LECTURE_CLARIFICATIONS_EN_RU.md for notation, collision, sentinel and complexity conventions. Required algorithmic methods are those explicitly listed above; no advanced DP optimization or linear-time suffix construction is required. Input-domain limits above define what the solution must support.

Appendix: lecture examples and conventions

- LCS prefix state: dp[i][j] concerns the first i and j characters. Compare a[i-1] and b[j-1] when i, j>0; zero row/column are base cases. One short lecture formula mixes index conventions.
- Interval merge uses the lecture 8 partition pattern. Its split cost is the sum of sizes in [l, r], whereas matrix-chain multiplication uses matrix dimensions. These cost rules are not interchangeable.
- Tree MWIS is the take/skip example from lecture 8. Choosing a root supports computation; it does not change the undirected problem or optimum.
- The file named Lecture 9 has a Lecture 10 title and an unrelated Kosaraju slide. The string-matching content is relevant; the extra graph slide adds no A4 requirement.
- The naive-search pseudocode must compare character indices j=0..m-1, not 0..m inclusive. Test starts i=0..n-m when m<=n. Handle an empty pattern separately according to A4's n+1-boundary rule.
- The intentionally small hash modulus 101 is for collision testing. Equal hashes need character verification. Exact RK has a candidate-sensitive bound and may take quadratic time.
- Suffix examples in lecture 10 use banana$ with an added sentinel. A4's output has exactly the nonempty suffixes of the original text; remove any internally added sentinel from output.
- The Aho example for text ushers in lecture 10 has incorrect occurrence positions. With zero-based inclusive start/end indices: she is 1..3, he is 2..3, hers is 2..5. The patterns he and she end at the same character; hers ends later. A start index is not an end index.
- Doubling with comparison sorting is $O(n \log^2 n)$; linear-time sorting of rank pairs per round permits $O(n \log n)$. Kasai is $O(n)$.
- DP does not universally turn exponential algorithms into polynomial time in bit-length: knapsack $O(nW)$ is pseudopolynomial. Top-down versus bottom-up performance depends on visited states, storage and implementation.